(a)

In [1]:
```python
import numpy as np

def f(x):
    return x**2 - np.exp(-x) - 2 * np.sin(x)

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("The function must have opposite signs at a and b.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        if f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2


root = bisection(f, 0, 2, 1e-6)
print("Root of f(x): ", root)
```

```
Root of f(x):  1.489588737487793
```

(b)

In [2]:
```python
import numpy as np

def f(t):
    return 10 * t - 4.9 * t**2

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("Bisection method fails. f(a) and f(b) must have opposite signs.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        elif f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2
```

```
root = bisection(f, 1, 100, 1e-6)
print("Time take projectile returns to initial height:", root, "seconds")
```

```
Time when projectile returns to initial height: 2.040816044807434 seconds
```

(c)

In [5]:
```
import numpy as np

def f(x):
    return x**2 - 3

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("Bisection method fails. f(a) and f(b) must have opposite signs.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        elif f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2

root = bisection(f, 1, 2, 1e-6)
print("Square root of 3 is:", root)
```

```
Square root of 3 is: 1.732050895690918
```

(d)

In [9]:
```
import numpy as np

def f(L):
    return L - (100*9.8)/(4*np.pi*np.pi)

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("Bisection method fails. f(a) and f(b) must have opposite signs.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        elif f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2
```

```
root = bisection(f, 1, 100, 1e-6)
print("Length of the pendulum for T=10 :", root, "meters")
```

```
Length L for T=10 : 24.82368966192007 meters
```

(e)

In [2]:
```
import numpy as np

def f(t):
    return 5 * (1 - np.exp(-t / 10)) - 2.5

def bisection(f, a, b, tol):
    if f(a) * f(b) >= 0:
        print("f(a) and f(b) must have opposite signs.")
        return None

    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(c) == 0:
            return c
        elif f(a) * f(c) < 0:
            b = c
        else:
            a = c
    return (a + b) / 2

time = bisection(f, 0, 50, 1e-6)
print("Voltage across the capacitor :", time, "V")
```

```
Voltage across the capacitor : 6.931471079587936 V
```

In [ ]:
```
```